



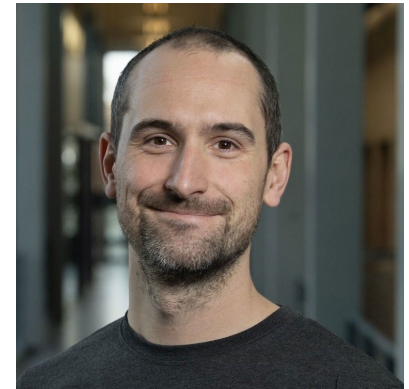
AGILE SOFTWARE ENGINEERING CONFIGURATION MANAGEMENT & DEVOPS

JOHN STOUBY PERSSON

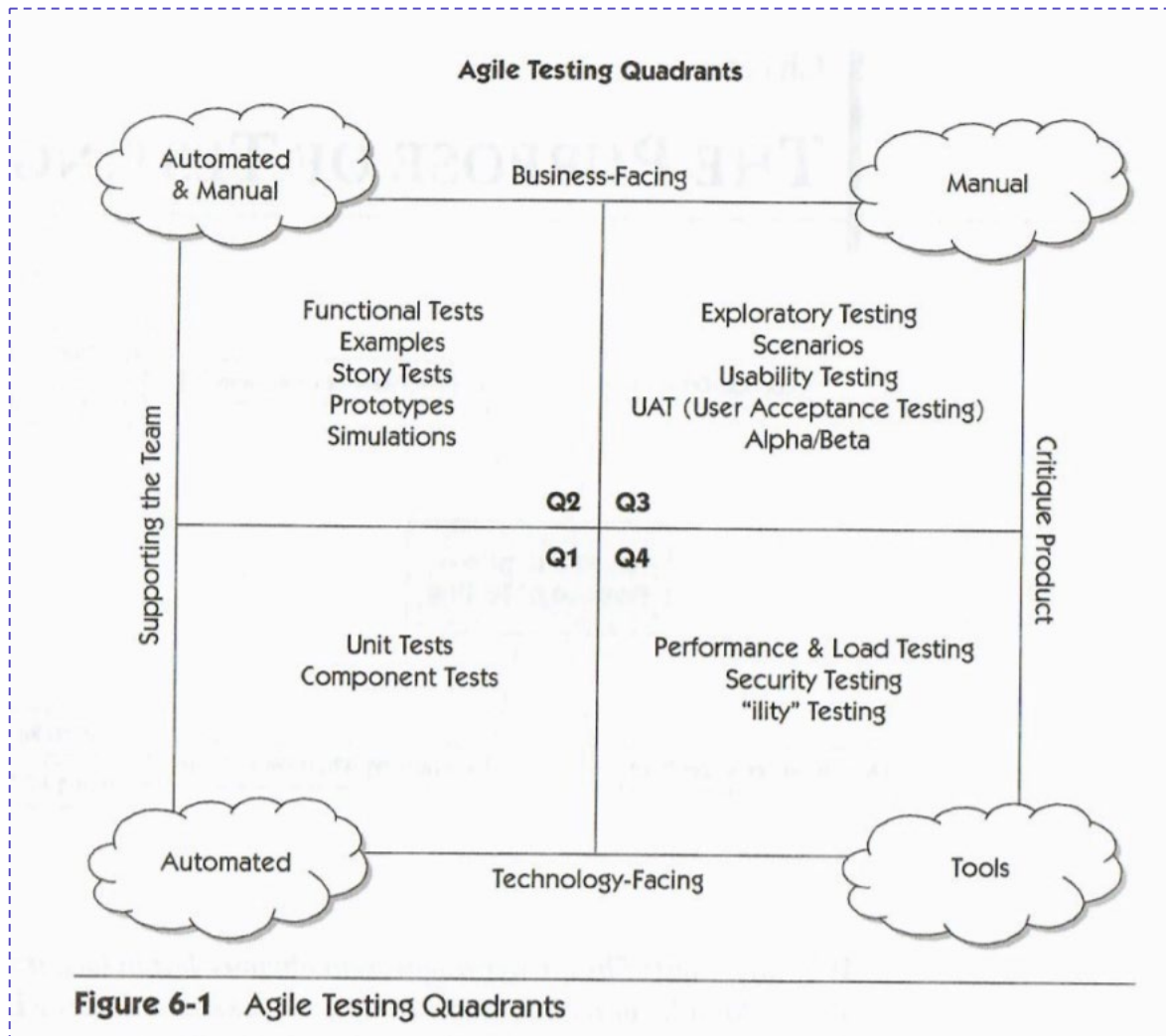


AALBORG UNIVERSITY
DENMARK

Test Management at RTX



Michael Kvist Svangren
SQA & Site Manager



Lecture objectives

Knowledge about the software engineering problems that configuration management and DevOps help to solve.

Skills in defining and managing processes for changes, versions, builds, and releases of software.

Competency to manage software configurations.

Configuration Management (CM)

... is concerned with the policies, processes, and tools for managing changing software systems.

Configuration: *“a collection of specific versions of hardware, firmware, or software items combined according to specific build procedures to serve a particular purpose”*

Involves four closely related activities:

- 1. Change management**
- 2. Version management**
- 3. System building**
- 4. Release management**

- Plan-driven vs agile CM
- Key terms: Branch, Merge, Baseline (not as defined by GIT!)

1. Change Management

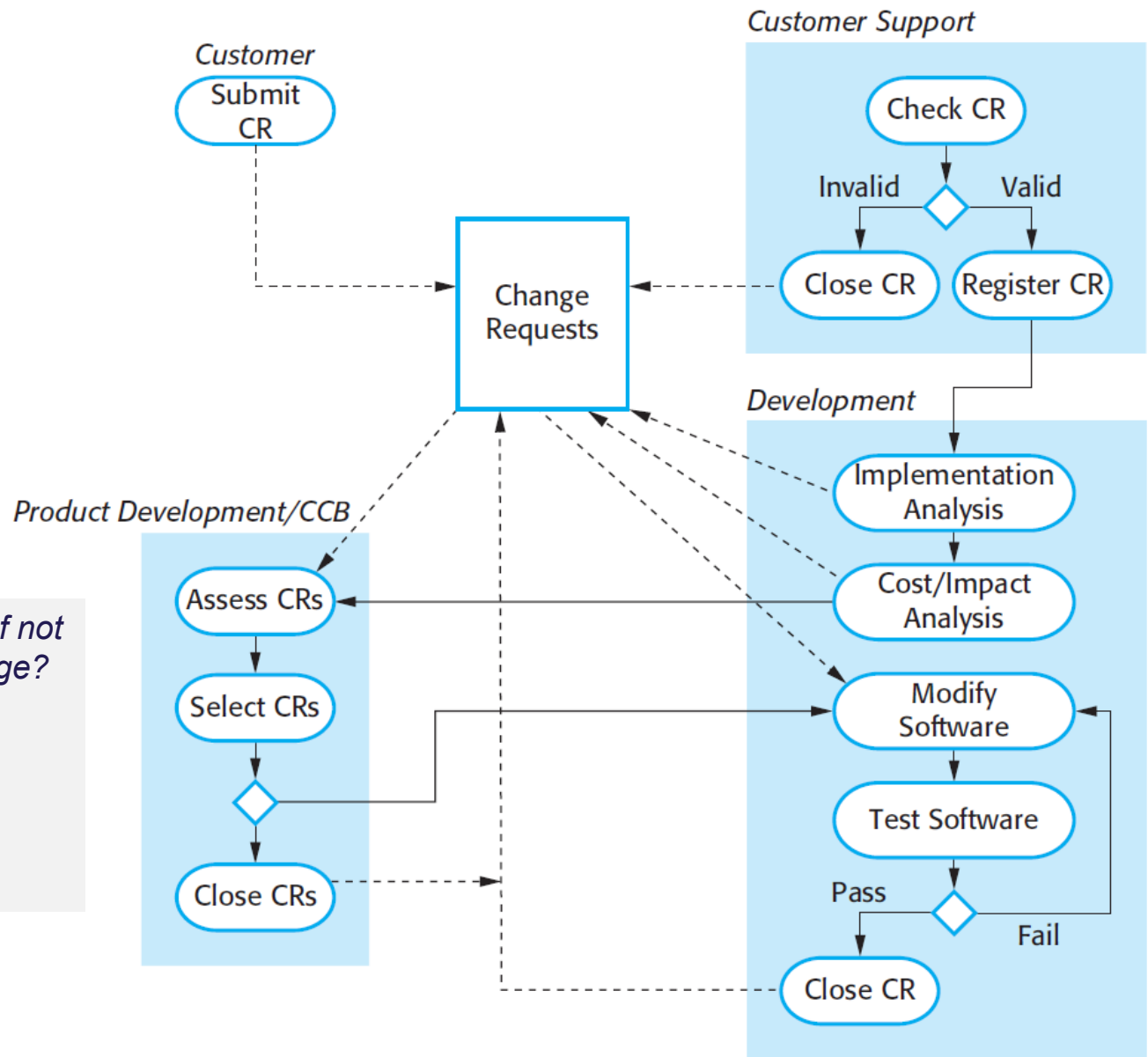
... Change is inevitable.

1. Keep track of requests for change to software from customers and developers
2. Work out the cost and impact of making these changes
3. Decide if and when changes should be implemented

Often submitted in a change request form.

Changes are managed by a Change Control Board (CCB) or product development group.

1. Change management: the process

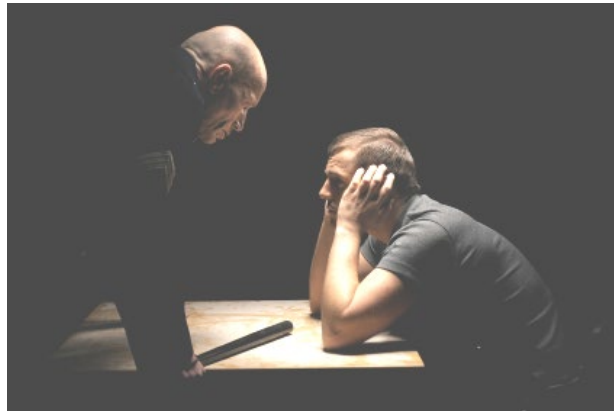


1. *Consequences of not making the change?*
2. *Benefits?*
3. *Users affected?*
4. *Costs?*
5. *Release cycle?*

1. Change management: problems and solutions

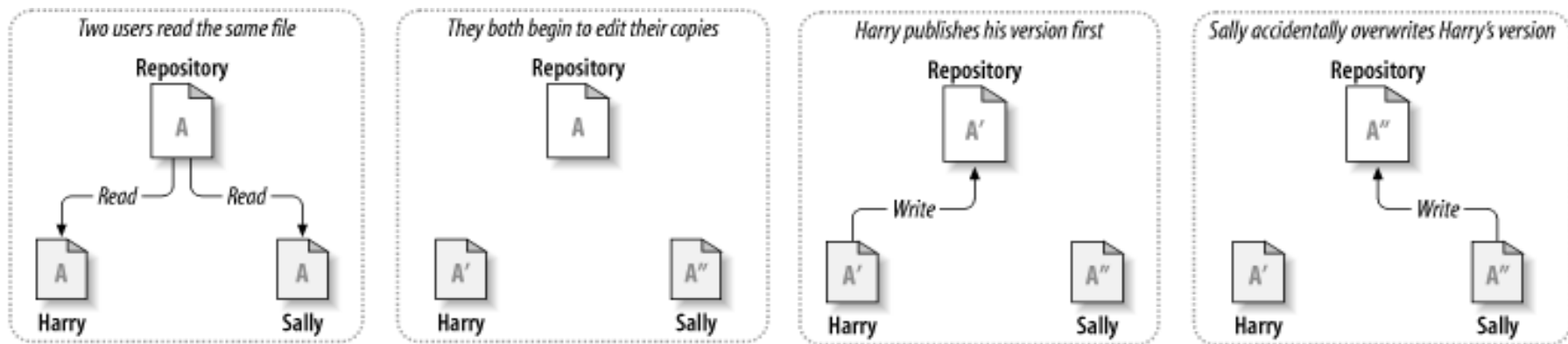
Identify two problems with how the person next to you (mis)manage configurations of code and text (or simply changes) in his/her projects.

Propose one improvement initiative that may help address one or more of these problems.



2. Version Control

Problem: Two or more people need to work on the same file



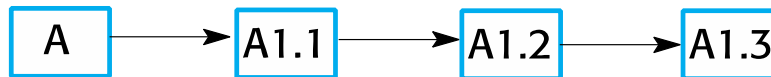
Solutions:

Pessimistic: file locking

Optimistic: version merging

2. Version Control: Codelines and baselines

Codeline (A)



Codeline (B)



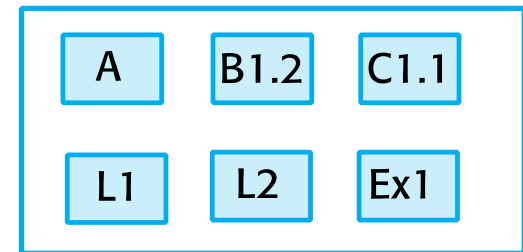
Codeline (C)



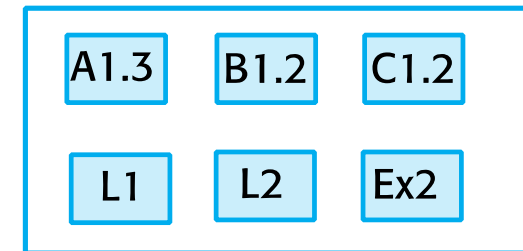
Libraries and external components



Baseline - V1

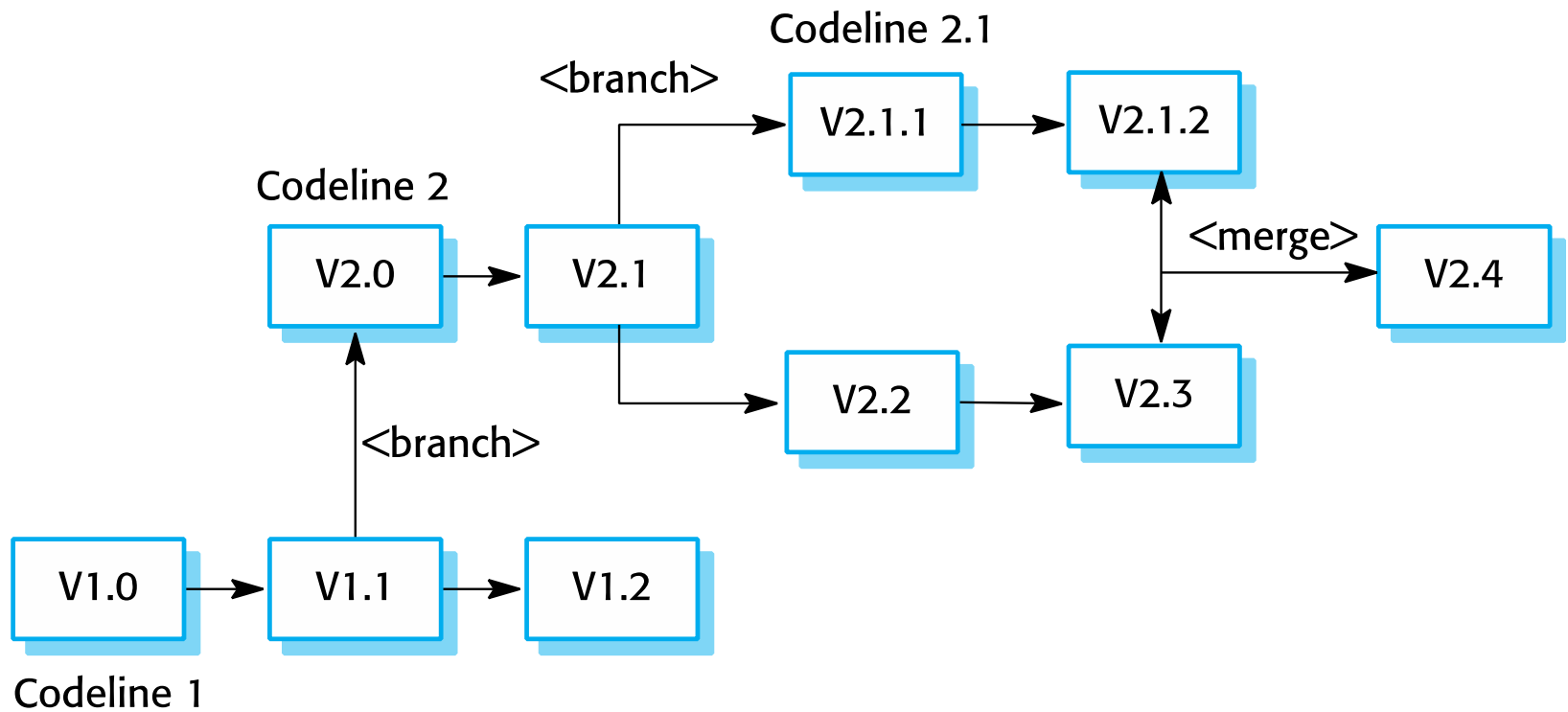


Baseline - V2



Mainline

2. Version Control: Branching and merging



Configuration Management terminology (Differs from Git)

Term	Explanation
Baseline	A baseline is a collection of component versions that make up a system. Baselines are controlled, which means that the versions of the components making up the system cannot be changed. This means that it is always possible to recreate a baseline from its constituent components.
Branching	The creation of a new codeline from a version in an existing codeline. The new codeline and the existing codeline may then develop independently.
Codeline	A codeline is a set of versions of a software component and other configuration items on which that component depends.
Configuration (version) control	The process of ensuring that versions of systems and components are recorded and maintained so that changes are managed and all versions of components are identified and stored for the lifetime of the system.
Configuration item or software configuration item (SCI)	Anything associated with a software project (design, code, test data, document, etc.) that has been placed under configuration control. There are often different versions of a configuration item. Configuration items have a unique name.
Mainline	A sequence of baselines representing different versions of a system.

CM terminology

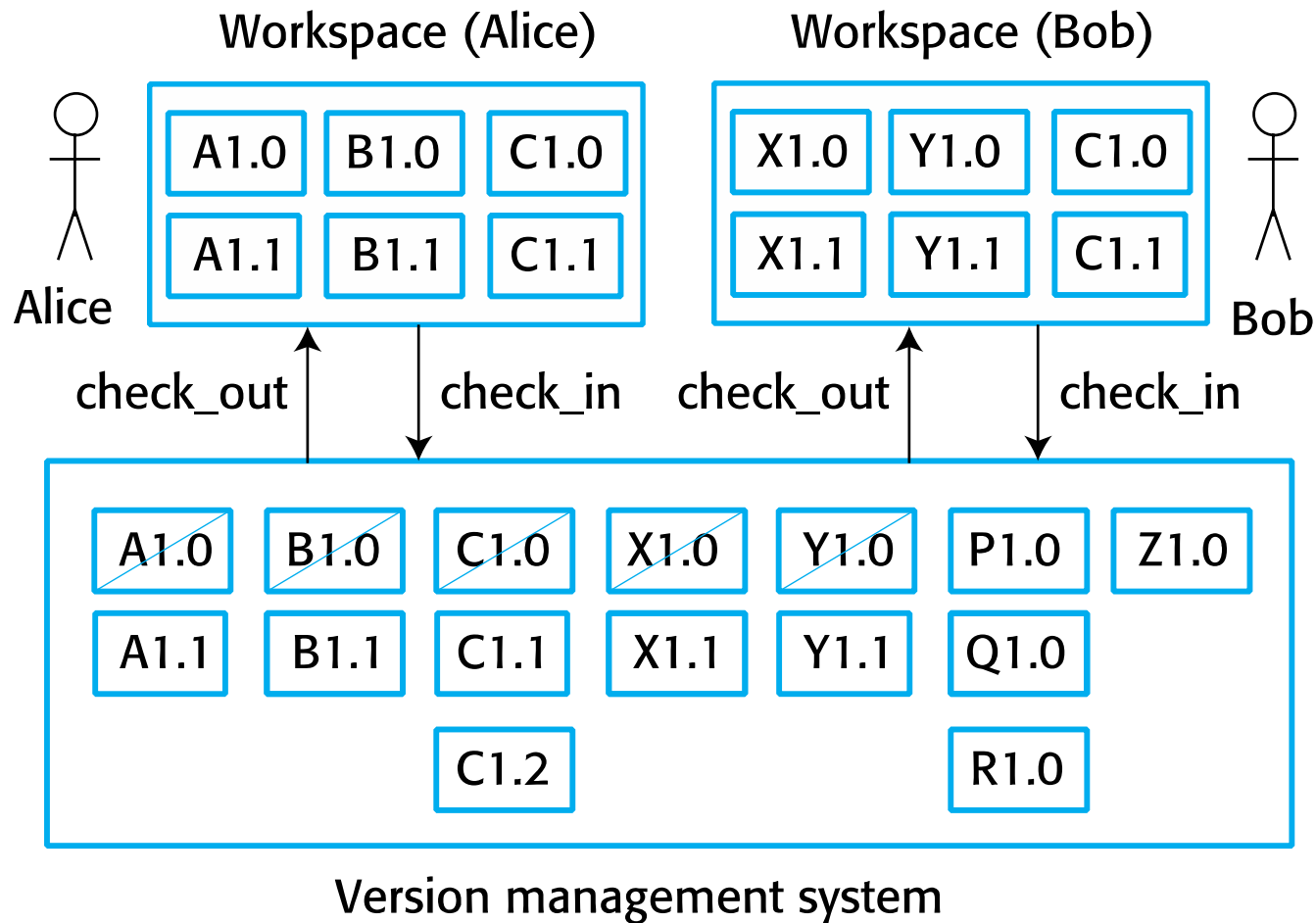
Term	Explanation
Merging	The creation of a new version of a software component by merging separate versions in different codelines. These codelines may have been created by a previous branch of one of the codelines involved.
Release	A version of a system that has been released to customers (or other users in an organization) for use.
Repository	A shared database of versions of software components and meta-information about changes to these components.
System building	The creation of an executable system version by compiling and linking the appropriate versions of the components and libraries making up the system.
Version	An instance of a configuration item that differs, in some way, from other instances of that item. Versions always have a unique identifier.
Workspace	A private work area where software can be modified without affecting other developers who may be using or modifying that software.

2. Version Control: Systems

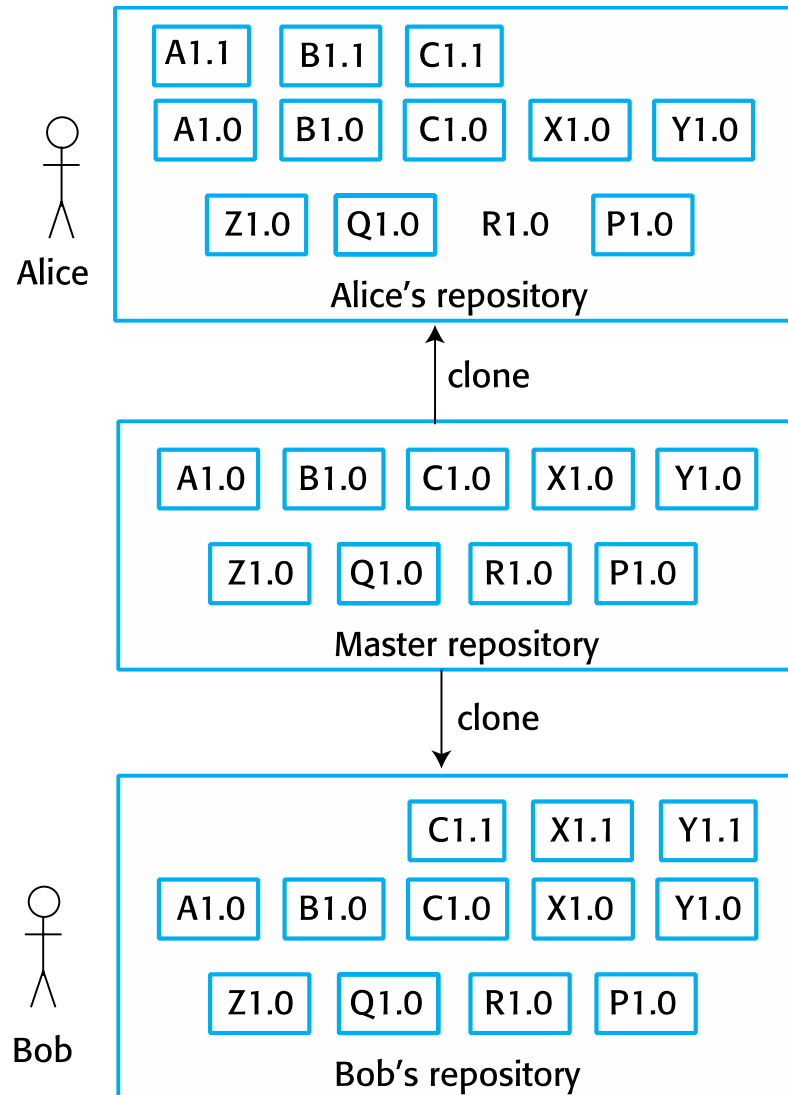
Version control (VC) systems identify, store and control access to the different versions of components. There are two types of modern version control system

- Centralized systems, where there is a single master repository that maintains all versions of the software components that are being developed. Subversion is a widely used example of a centralized VC system.
- Distributed systems, where multiple versions of the component repository exist at the same time. Git is a widely-used example of a distributed VC system.

2. Version Control: Centralized Check-in/Check-out



2. Version Control: Decentralized repository cloning



2. Version Control: Benefits of distributed version control

It provides a backup mechanism for the repository.

- If the repository is corrupted, work can continue, and the project repository can be restored from local copies.

It allows for off-line working so that developers can commit changes if they do not have a network connection.

Project support is the default way of working.

- Developers can compile and test the entire system on their local machines and test the changes that they have made.

Essential in open source development! Why?

3. System Building: Tools

Building a large system is
computationally expensive and
may take several hours

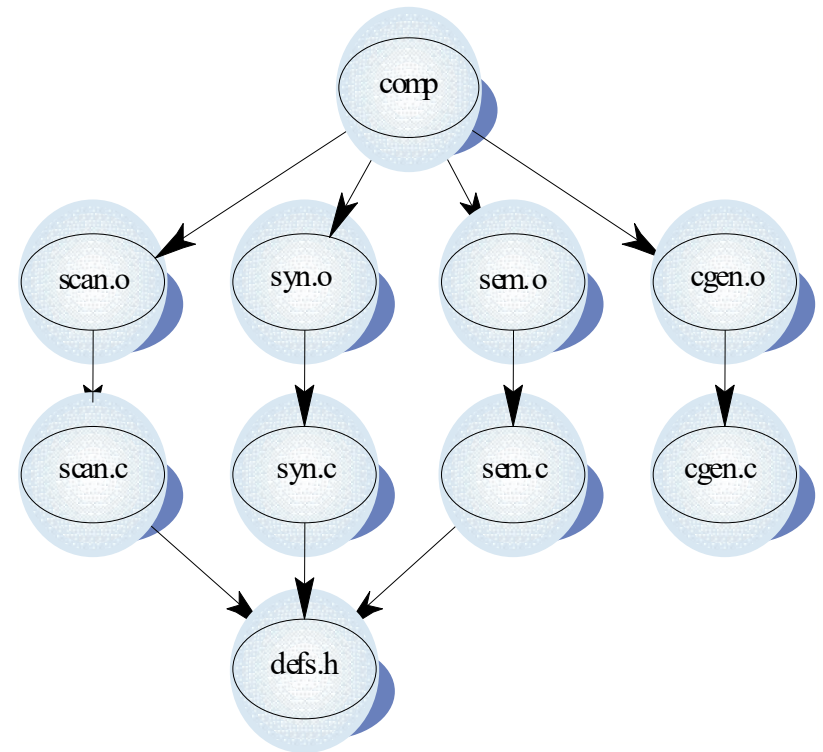
Hundreds of files may be involved

System building tools may provide:

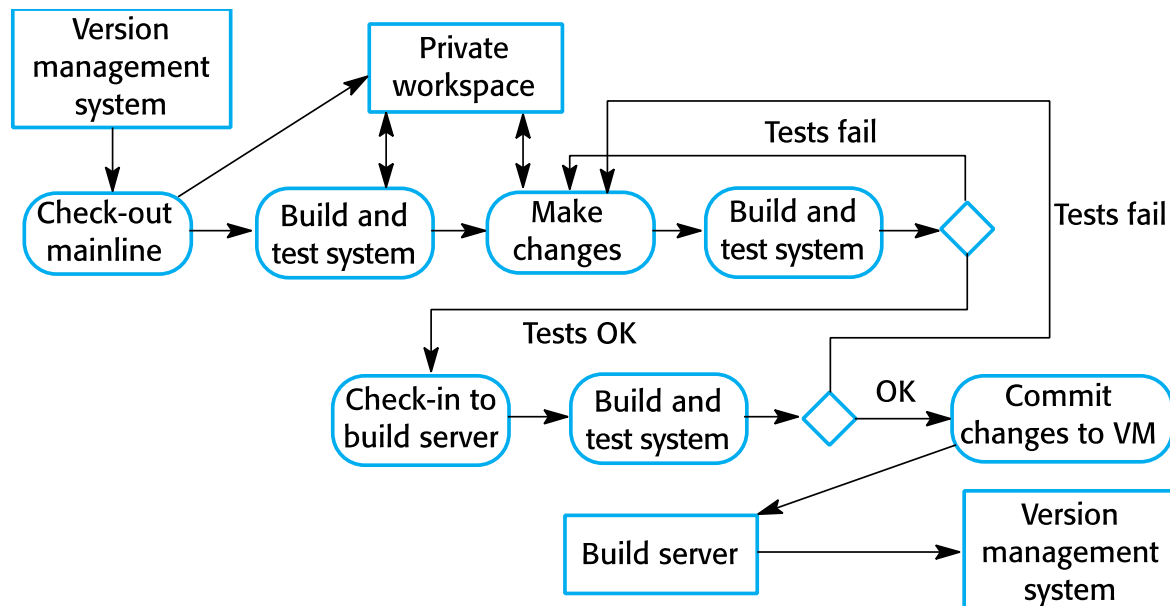
- a dependency specification language and interpreter
- tool selection and instantiation support
- distributed compilation

Examples:

[Scons](#), [Jenkins CI](#)



3. System Building: Continuous integration



3. System Building: Daily building or (weekly, monthly, ...)

The development organization sets a delivery time (say 2 p.m.) for system components.

- If developers have new versions of the components that they are writing, they must deliver them by that time.
- A new version of the system is built from these components by compiling and linking them to form a complete system.
- This system is then delivered to the testing team, which carries out a set of predefined system tests
- Faults that are discovered during system testing are documented and returned to the system developers. They repair these faults in a subsequent version of the component.

4. Release Management

... A system release is a version of a software system that is distributed to customers.

1. The release must be documented to ensure that it can be re-created
2. A release Baseline is that documentation. It includes all versions of codefiles, libraries, documentation as it was when the release was created

NOTE: This is the definition of "baseline" in context of configuration management. The word baseline is used in many other context, baseline cost in project planning. GIT also uses the term baseline for something else.

Plan-driven vs agile configuration management



Plan-driven configuration management

- Configuration management is the organizational and tool supported management of change to software products
- Formal document naming schemes and management in a database
- The configuration database should record information about changes and change requests
- A consistent scheme of version identification should be established using version numbers, attributes or change sets
- System releases include executable code, data, configuration files and documentation
- System building involves assembling components into a system
- Tools are available to support all CM activities
- Tools may be stand-alone or may be integrated systems which integrate support for version management, system building and change management

Agile development and CM

Agile development, where components and systems are changed several times per day, is impossible without using CM tools.

The definitive versions of components are held in a shared project repository and developers copy these into their own workspace.

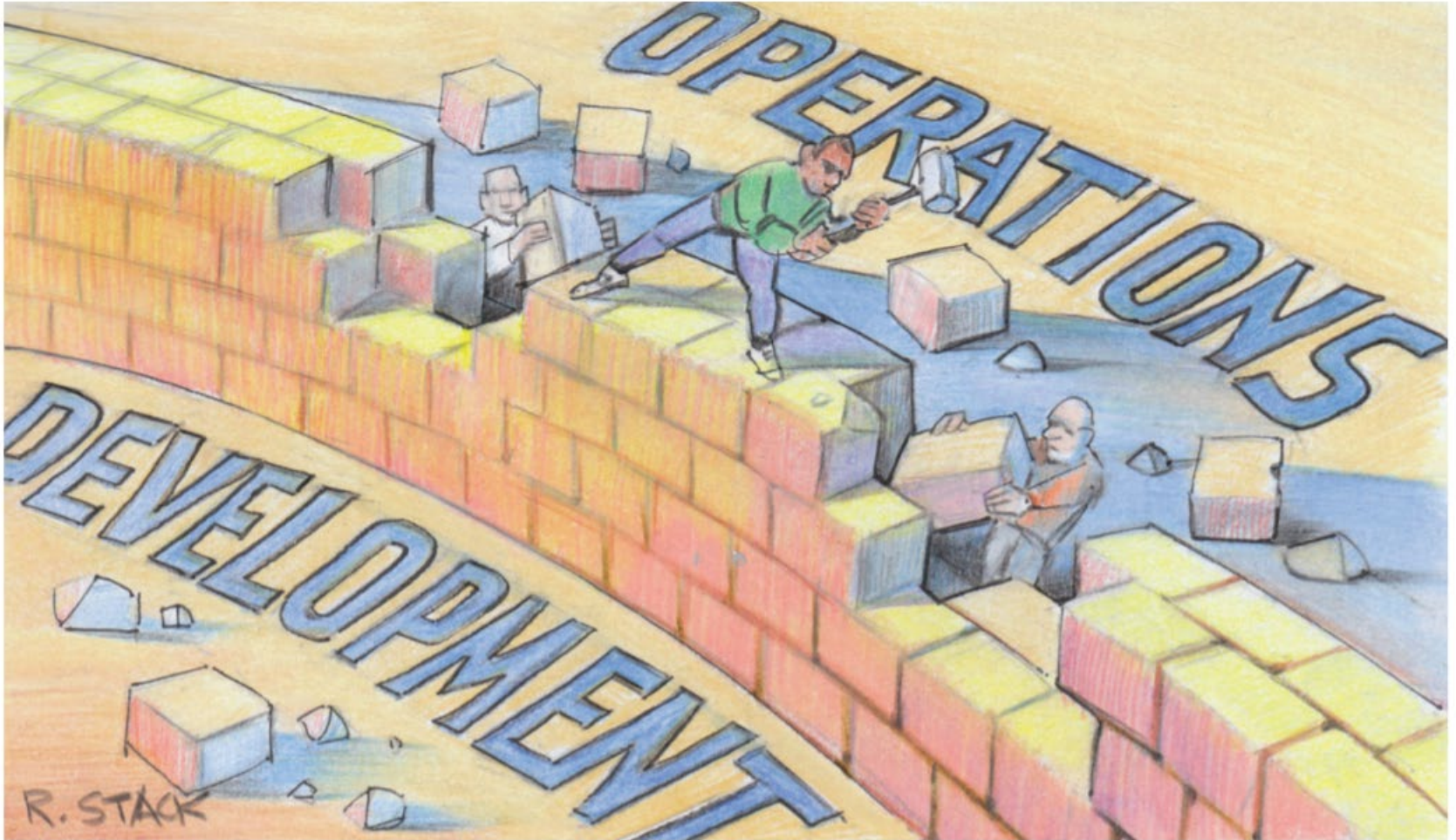
They make changes to the code then use system building tools to create a new system on their own computer for testing. Once they are happy with the changes made, they return the modified components to the project repository.

Plan-driven and agile configuration management

	Plan-driven	Agile
<i>Focus</i>	documents and code	code
<i>Activities</i>	CM practice, change management, version control, automated build	version control, automated build
<i>Responsible</i>	CM team, CM board	programmers
<i>Process</i>	formal, managed	informal and integrated with practice environment
<i>Outcome</i>	CM audit (documented product control)	next release
<i>Importance</i>	indispensible in medium and large projects	indispensible



DevOps



DevOps

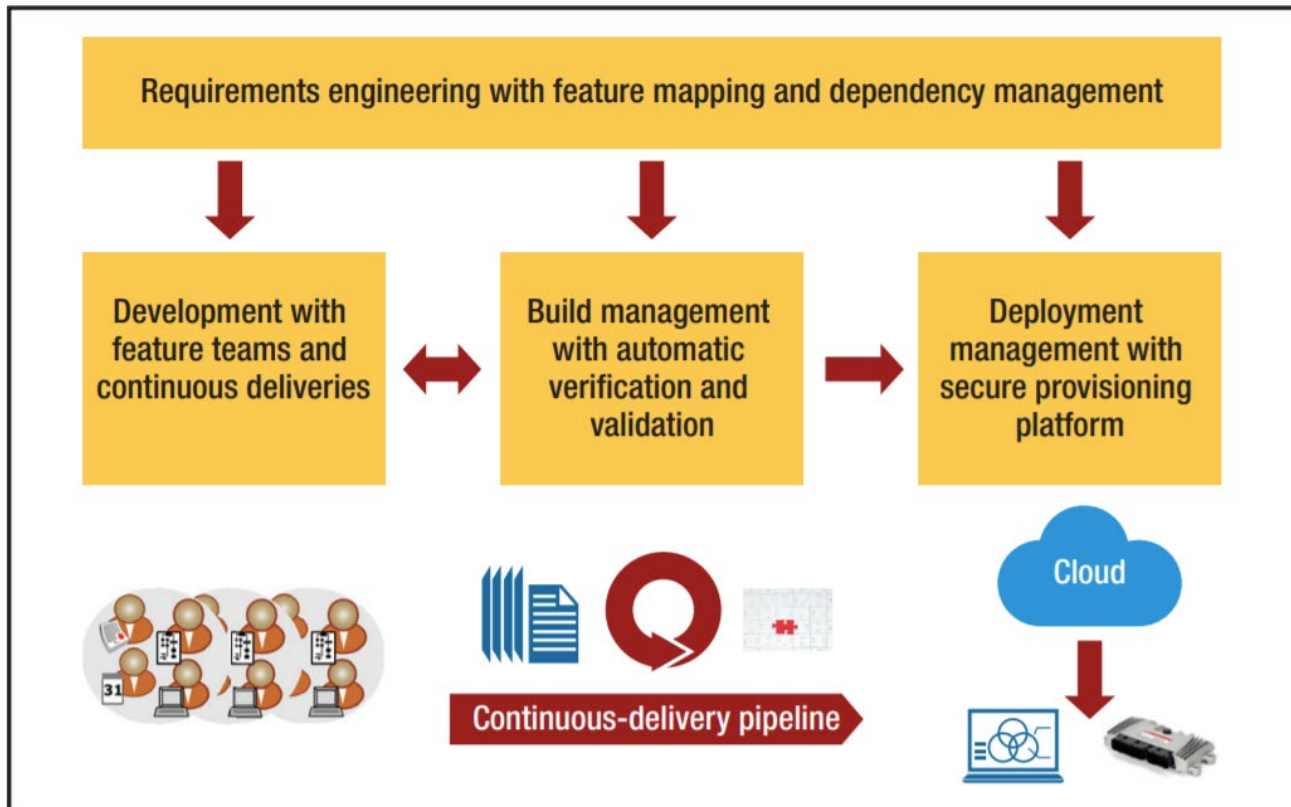


FIGURE 1. The generic DevOps production and delivery process. It aims to better integrate the development, production, and operations business processes with adequate technology.

DevOps and LEAN process thinking

Value stream

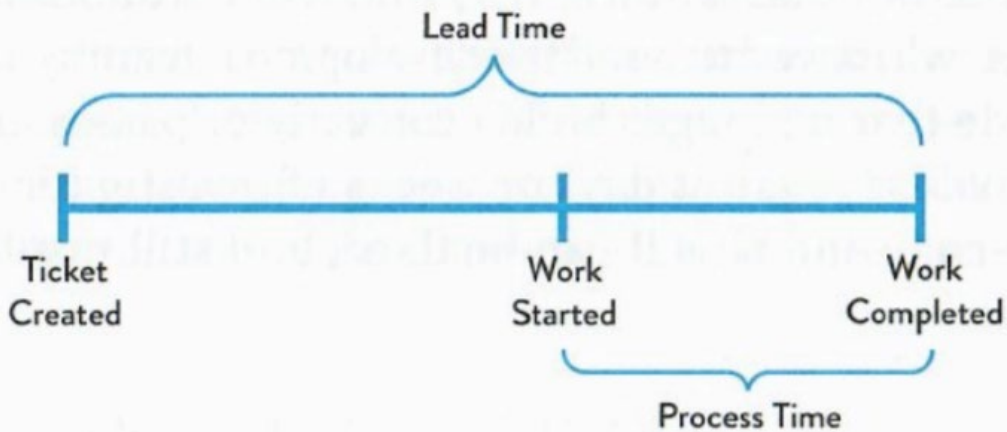
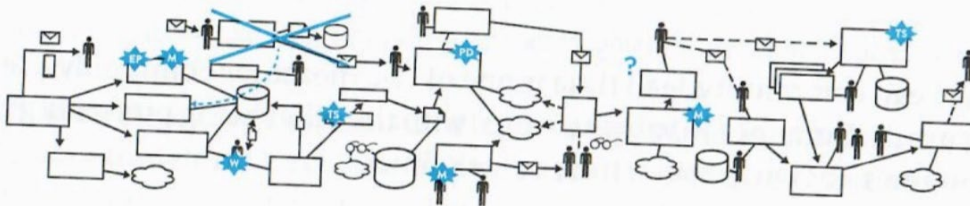
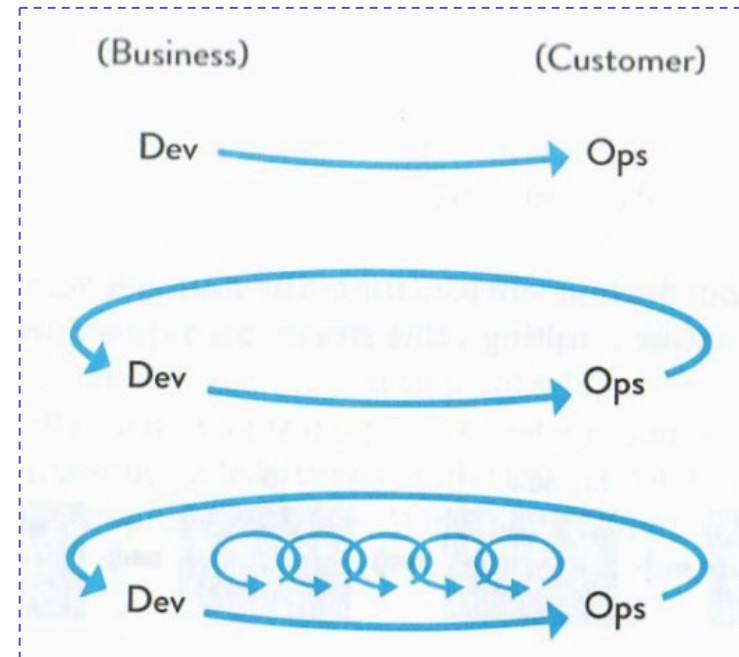


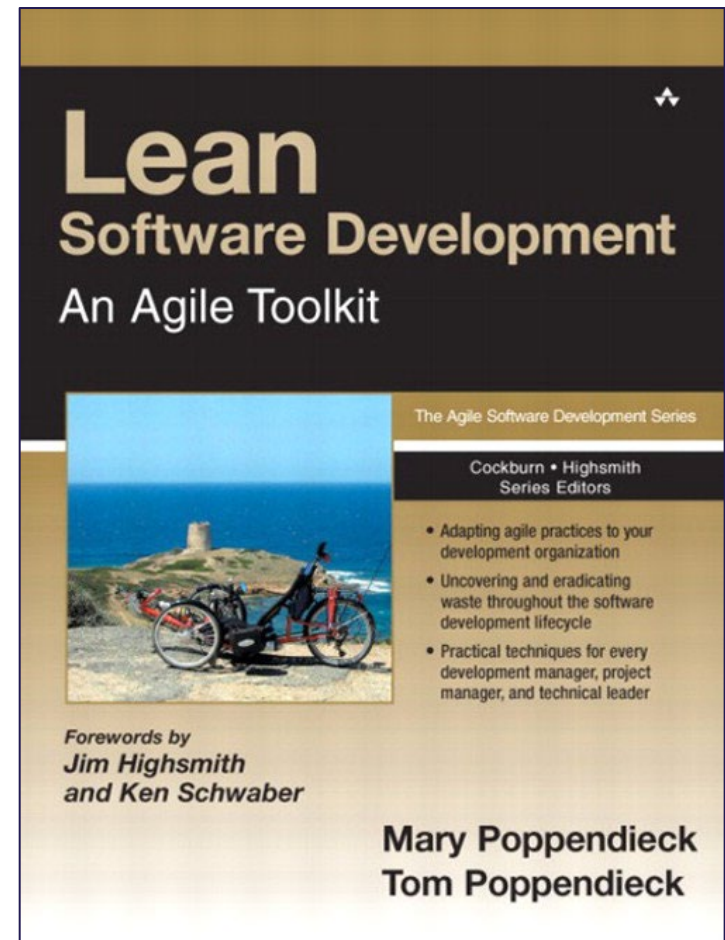
Figure 2. Lead time vs. process time of a deployment operation



Lean Software Development

Seven principles

- optimize the whole
- eliminate waste
- build quality in
- learn constantly
- deliver fast
- engage everyone, and
- keep getting better.



[Poppendieck, M., & Cusumano, M. A. \(2012\). Lean software development: A tutorial. *IEEE Software*, 29\(5\), 26-32.](#)



Lean Software Development



Old/new principle	Myth	Lean
Eliminate waste	Early specification reduces waste	Research shows that 64% is waste
Build integrity in (build quality in)	Make it completely right the first time	Make it better everytime
Amplify learning (learn constantly)	Predictions create predictability	Predictions are not facts
Deliver fast	Haste is waste	Speed is a parameter in competition
Empower the team (engage everyone)	There is one best way!	There are many best ways, to be found by “respecting, supporting and allowing” people
Defer commitment (keep getting better)	Decisiveness is a virtue	Defer commitment
Optimize the whole	Optimize through decomposition	A system is not the sum of its parts – it is the products of their interactions

Group exercises

Exercise 1: Configuration management in your project.



Lecture objectives

Knowledge about the software engineering problems that configuration management and DevOps help to solve.

Skills in defining and managing processes for changes, versions, builds, and releases of software.

Competency to manage software configurations.